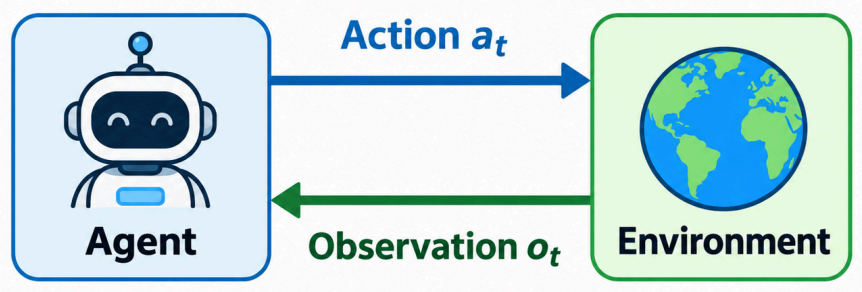
EmbodiedAI Week 7: LLM Agent

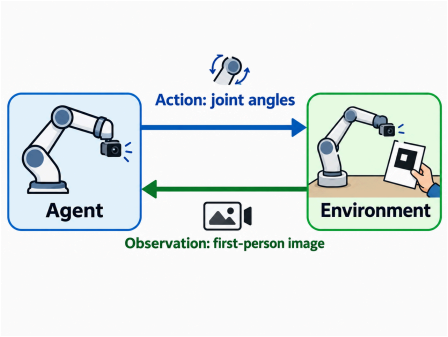
一、什么是 Agent?

> 🤖 *“Agent 是任何能够通过传感器感知环境，并通过执行器对环境采取行动的实体。”*
>
> (原文：*An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.*)
>
> —— Russell, S. J., & Norvig, P. (1995). Artificial Intelligence: A Modern Approach.

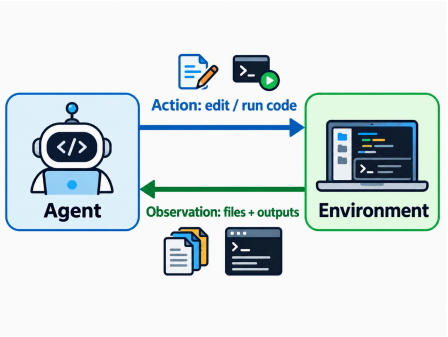


在数学上，我们可以把 Agent 的决策过程描述为： $a_t = \pi(o_{0:t})$

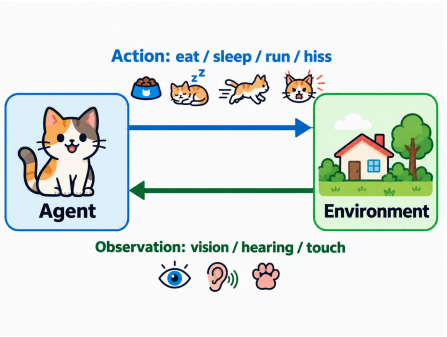
因此，下面这些都是 Agent:



我们上节课做的视觉伺服机械臂



跑在你电脑里的 Coding Agent



爱猫人士家里养的猫

通常来说，Agent 的输入输出我们都能够用数学的形式定义。

例如对于上节课我们做的视觉伺服，Agent 的输入是一张 RGB 图像，可以写成一个张量 $o_t \in \mathbb{R}^{H\times W\times 3}$；输出是每个关节角的移动，可以写成向量 $a_t = [\Delta\theta_1, \Delta\theta_2, \ldots, \Delta\theta_6]$。

那么，问题就在于如何得到这个函数 π。想得到它，通常有两种思路：

1. **由人类手工设计** —— 这就是我们上节课的做法。

a. 我们手推了三维世界中的点如何被投影到相机成像平面、变成图像中的一个像素，像素坐标在相机运动下如何变化；以及机械臂关节角如何决定末端位姿（FK）、末端位姿和相机位姿之间是什么关系……

b. 最终，我们把推导出的像素坐标和关节角的数学关系写成程序，这个程序就是我们的 π 。

2. **从数据中学出来** —— 这就是机器学习的核心思想。

a. 我们会先定义一类候选函数。例如，假设我们认为 π 是一个线性函数：$a_t = A\mathbf{x}_t$ ，其中 $A \in \mathbb{R}^{6\times 3HW}$ 是一个矩阵；这样我们就确定了函数的形式，但还不知道矩阵 (A) 中每个元素应该是多少。

b. 然后我们要定义一个目标 $\mathcal{L}$ ，来评价每个函数的好坏。例如我们希望机械臂行动之后，目标点能够尽可能接近画面中心，那么我们就可以定义 $\mathcal{L} = |s_{t+1} - s^*|^2$ ，其中 s^* 是目标点的位置。$\mathcal{L}$ 越小，说明 A 越好。

c. 接下来，我们可以让机械臂根据当前的矩阵 A 执行动作，观察这些动作对环境产生的影响。通过不断尝试和调整矩阵 A ，我们希望找到一个使目标函数 $\mathcal{L}$ 在这些实验中尽可能小的矩阵。

即：$A^* = \arg\min_A \frac{1}{N}\sum_{i=1}^{N} \|s_{i,t+1}(A) - s_i^*\|^2$

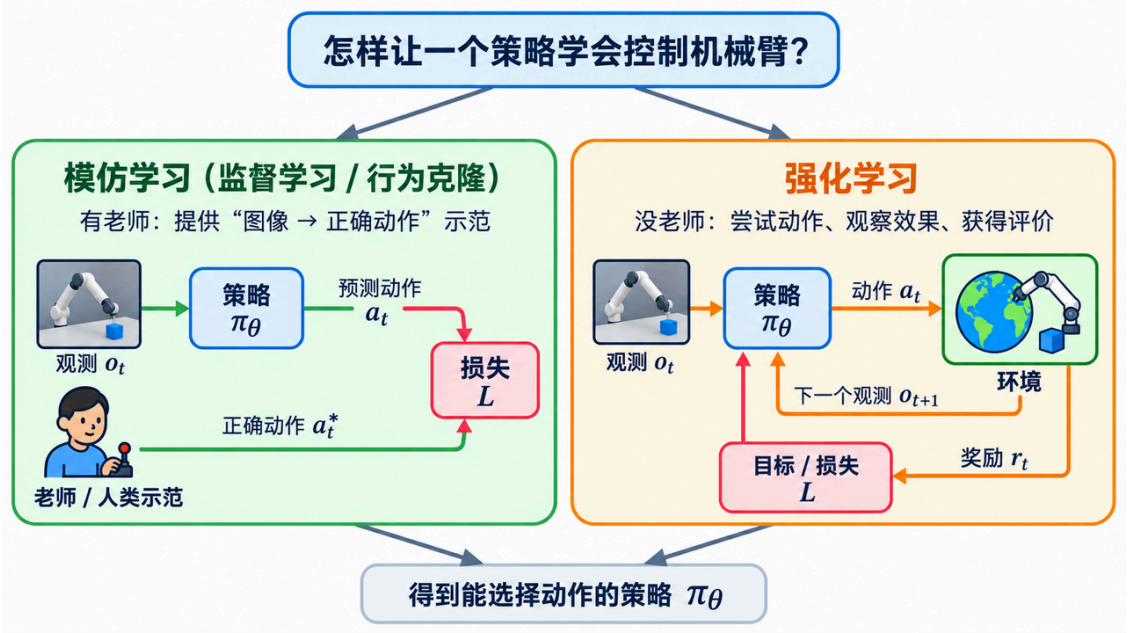
这两条路线并不矛盾，也可以结合起来。我们不必让模型把已经知道的规律重新猜一遍：相机如何成像、机械臂各关节如何连接、关节能够转动到什么范围，都可以直接写进系统；再让模型从数据中学习摩擦、标定误差等难以准确描述的部分。

我们称这些“在通过交互形成经验之前，就假设世界上普遍存在的规律”，叫做**先验知识（prior knowledge）**。

先验用得合适，模型通常能用更少的数据学得更快；但如果先验本身不准确，也可能限制模型。

二、模仿学习与强化学习

在我们框定了一类候选函数后，如何定义这个目标 $\mathcal{L}$ ？



2.1 从示范动作学习：模仿学习

假设一个熟练的操作员手动控制机械臂，每当机械臂看到一幅图像，操作员就告诉它：“此时关节应该这样动。”我们可以采集数据：$\mathcal{D} = \{(o_i, a_i^*)\}_{i=1}^N$，其中 a_i^* 表示第 i 个观测对应的示范动作。这样一来，$\mathcal{L}$ 自然就可以定义为函数产生的动作 $\pi_\theta(o_i)$ 和操作员给出的 a_i^* 的差异：

$$\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|\pi_\theta(o_i) - a_i^*\|_2^2$$

这叫作**行为克隆（Behavior Cloning, BC）**，是模仿学习的一种简单形式。因为训练样本中有明确的正确输出，它也可以看作监督学习。

- **好处：**不必让机器人进行大量危险试错；只要示范足够好、覆盖足够广，就能学到有用行为。
- **困难：**模型遇到示范中没有出现过的情况时，可能不知道怎么办。

2.2 从环境反馈学习：强化学习

现在把老师撤走。我们只告诉机器人：“让目标一直保持在画面中心。”没有人提供每一步正确的关节指令。机器人采取动作后，目标可能更靠近中心，也可能更远；我们用**奖励（Reward）**评价这个结果，例如：

$$r_t = -\|s_{t+1} - s^*\|_2^2$$

结果越接近目标位置 s^*，奖励越高。但只看当前一步还不够：某个大动作也许瞬间对准了目标，却让后续控制变得困难。因此我们需要优化一段时间内的累积奖励：

$$\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$$

这里，r_t 是第 t 步的奖励，γ 控制我们有多重视未来。奖励不是“标准答案动作”：Agent 必须尝试动作、观察后果、积累经验，再逐步找到更好的策略。从环境返回的奖励中学习，就是强化学习。

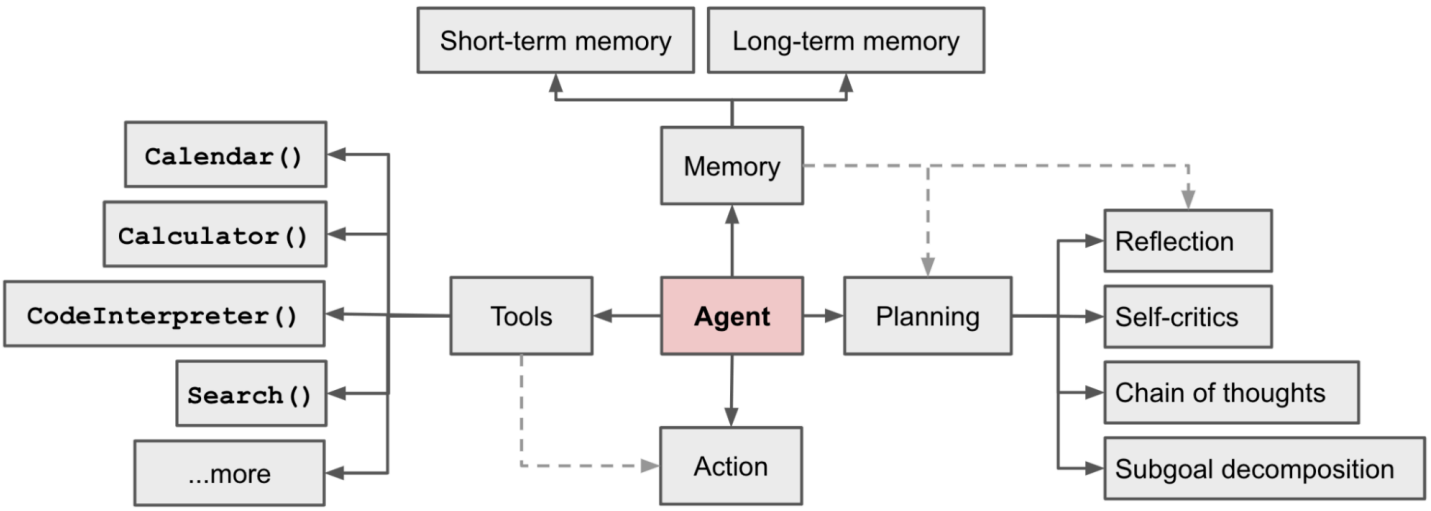
2.3 理解开放任务

如果任务不再只是“追踪固定目标”，而是“找出我刚才用过的东西”或“把蓝色物体移到红色物体左侧”，策略还需要理解任务目标 g：

$$a_t = \pi_\theta(o_{0:t}, g)$$

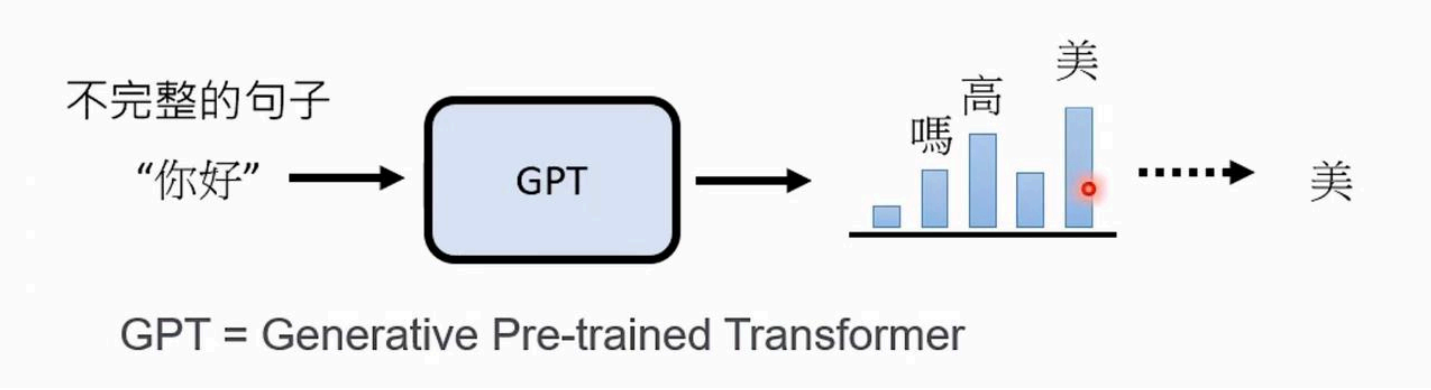
可是，g 可能是一句开放的自然语言。什么模型能够理解如此丰富的指令？下面我们把视线转向大语言模型。

三、LLM Agent



LLM Agent 本质上是赋予 LLM 感知、记忆、规划和执行动作的能力

3.1 回忆一下LLM



大语言模型是一个输入文本、输出文本的模型。它被训练做的任务是文字接龙 —— 输入一段不完整的文本，输出这个文本的下一个 token（可能是汉字、单词或者单词的一部分，例如“ful”、“ly”）。

因为它在海量的文本上被训练过，所以它知道“现在的美国总统是谁”后面应该接“特朗普”，也知道“请帮我写一段代码实现计算器功能”后面应该接什么代码。

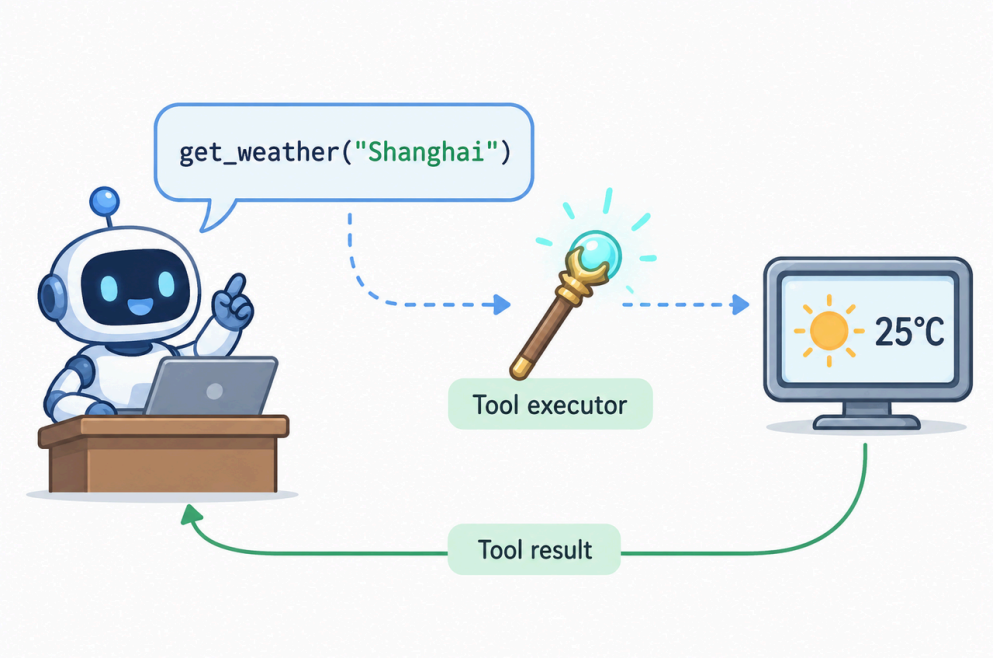
—— 但是，只能输出文本的 LLM，怎么能产生一个“动作”，对环境造成影响？

3.2 Tool Using

LLM 原本只会输出文本。要让它真的查询天气、读取相机或控制机械臂，需要把模型输出转换为程序能够执行的工具调用。

3.2.1 Function Calling：让模型提出结构化调用请求

可以把工具想象成一根魔杖，把函数调用想象成事先约定好的咒语。LLM 负责根据任务选择咒语；真正执行操作的是那根魔杖，也就是 Agent 程序和工具。



LLM 产生工具调用请求，工具执行程序完成操作，再把结果返回给模型。

例如，当用户问“上海天气怎么样”时，模型可以产生这样的输出：

结构化工具调用请求

```
{
  "name": "get_weather",
  "arguments": {"city": "Shanghai"}
}
```

得到这些“咒语”之后，我们的程序解析出 LLM 需要调用获取天气信息的工具、并且传入的参数是“上海”。然后我们的程序会执行天气查询的功能，将上海的天气返回给 LLM。



Function Calling 的五个环节：用户提问、模型生成请求、程序执行、工具返回结果、模型组织回答。

1. 开发者预先声明有哪些工具、每个工具做什么、需要哪些参数。相当于给 LLM 写一个说明书。
2. LLM 根据问题决定是否使用工具，并参考“说明书”生成调用请求。
3. Agent 程序检查工具名称、参数与权限，再执行真实函数。
4. 程序把执行结果返回给 LLM。
5. LLM 结合结果生成最终回答。

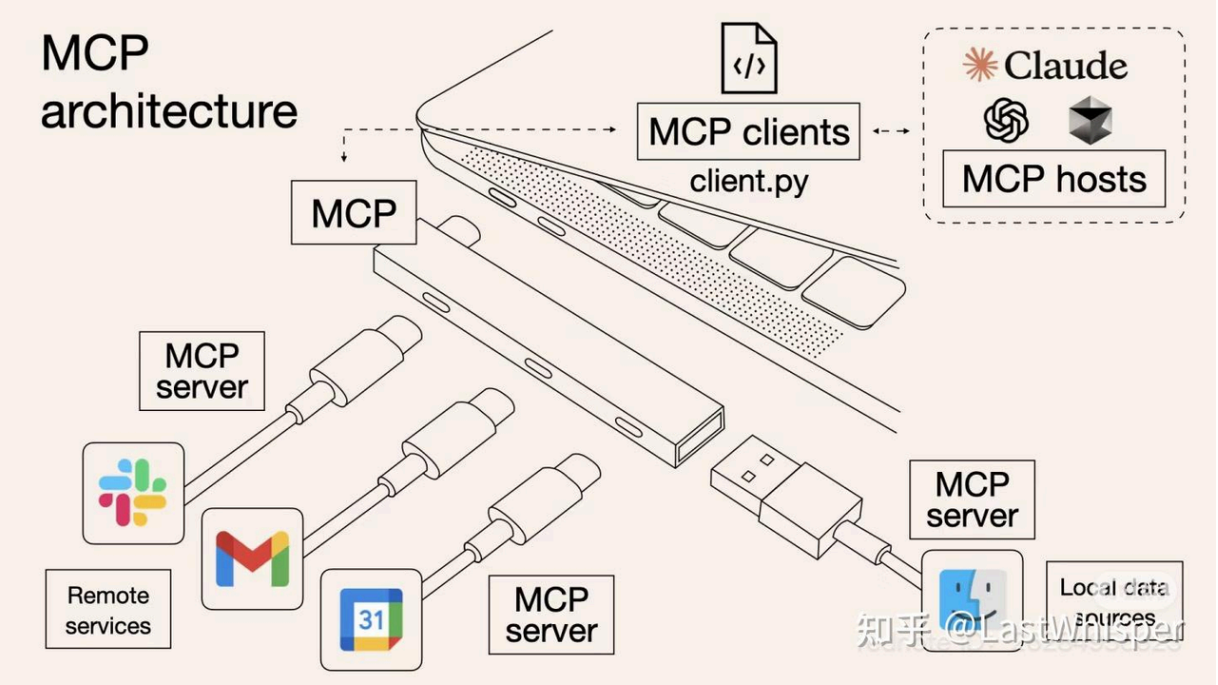
例如我们想让 LLM Agent 自动调用我们机械臂上的视觉伺服功能，我们只需把它上节课的视觉伺服程序包装成 Tool，让 LLM 自己决定何时启动、追踪哪个目标以及何时停止。

3.2.2 MCP：调用外部工具的标准协议

通常来说，我们不会希望我们的 Agent 只会调用“天气预报”这一个功能。相反，我们希望这个世界上一切能够调用的外部工具，从赛博世界的搜索、查询、回复邮件；到真实世界的买票、订餐、操控机器人，一切的外部工具都能为我们所用。

但如果按照 function calling 的方式，我就要自己搜罗工具、定义它的调用方式并描述给 LLM，还要完成工具调用的代码，非常麻烦。

我能不能把它做成一个“应用商店”一样的地方，让所有外部工具的供应商用统一的格式来提供各自的服务，再用统一的规范来管理这个服务被发现、被调用、返回结果的流程，使得任何一个 LLM Agent 都能够轻松接入这些工具？



—— 这就是**MCP（Model Context Protocol，模型上下文协议）**所做的事情。MCP 提供了一种统一连接外部工具、数据和工作流程的方式。它定义了三个角色：

- **Host：** 运行整个 Agent 的应用程序，负责管理模型、工具及其执行过程。
- **Client：**Host 中负责与 MCP Server 建立连接、发送请求和接收结果的组件。
- **Server：**向外部提供工具的具体功能和信息的程序。

Function Calling 解决的是模型如何表达“我要调用哪个工具、传入什么参数”；MCP 解决的是“Agent 应用如何以统一的方式发现、连接和使用外部能力”。

3.2.3 Agent Skills：有工具不等于会完成任务

俗话说“差生文具多”，前面的 MCP 大大丰富了我们的文具库，但是 LLM 仍然可能完全不会用。



这个时候就需要我们给 LLM Agent 编写一份 **Skill**。

Tool 描述 Agent “能够做什么”；Skill 则告诉 Agent “什么时候做、按什么顺序做、怎样判断完成”。skill 通常是一份 Markdown 文档，我们可以理解成我们给 LLM Agent 写的“操作指南”，它可以包含完成某类任务的步骤、经验、注意事项，以及相关代码和参考资料。

假设我的 LLM 通过 MCP 获得了三个外部工具：锤子、螺丝刀、开瓶器。

现在我告诉 LLM：请你帮我组装一个凳子。

如果 LLM 不知道组装凳子的规范流程，即便是我提供了这些工具以及对应的使用说明，可能它还是做不好一张凳子。这个时候我可能会写这样一个 skill：

代码块

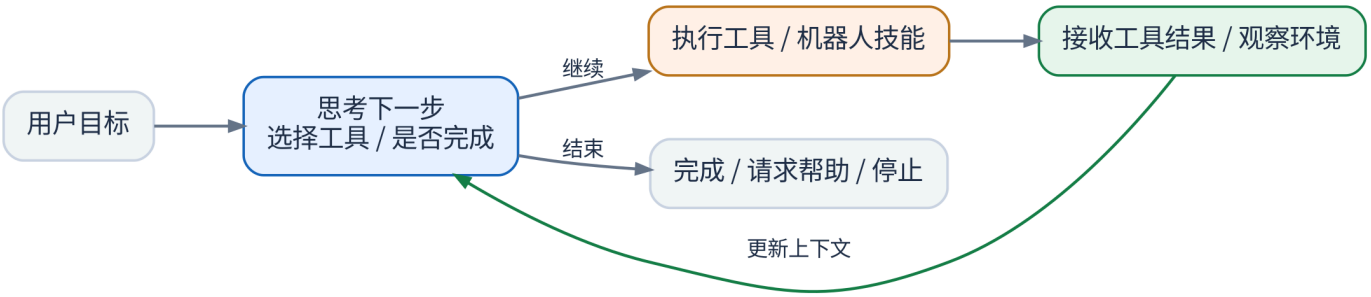
```
首先检查所有木板、螺丝和连接件是否齐全。
阅读组装说明，确定零件之间的连接关系。
根据连接件类型，选择螺丝刀或锤子进行组装。
每完成一个步骤，检查零件位置和连接是否正确。
最后检查木凳的结构是否稳固；如果发现问题，重新检查相关步骤。
```

每当涉及到组装相关的任务，LLM Agent 就会调出这个 skill，按照上面指定的步骤和规范执行动作。

3.3 Agent Workflow

复杂任务通常不是进行一次输出、或者调用一次工具就能完成。就像我们平时写代码一样，我们会频繁地遇到问题，就需要反复地分析问题、调整计划、解决问题……通常在多次迭代之后才会收敛到最后的结果。

同样，Agent 也需要根据每次执行结果动态调整计划、总结经验、持续推进直到完成，这就是 **Agent Loop**。



Agent Loop：根据目标决策、调用工具、观察反馈，必要时返回决策环节。

1. **Observe：**读取当前观察、工具结果和已有上下文。
2. **Decide / Reason：**判断目标是否完成，并选择下一步。
3. **Act：**调用工具或执行动作。
4. **Feedback：**检查结果、更新任务状态；未完成则继续循环。

一次简化的执行轨迹可能是：

目标追踪的 Agent Loop

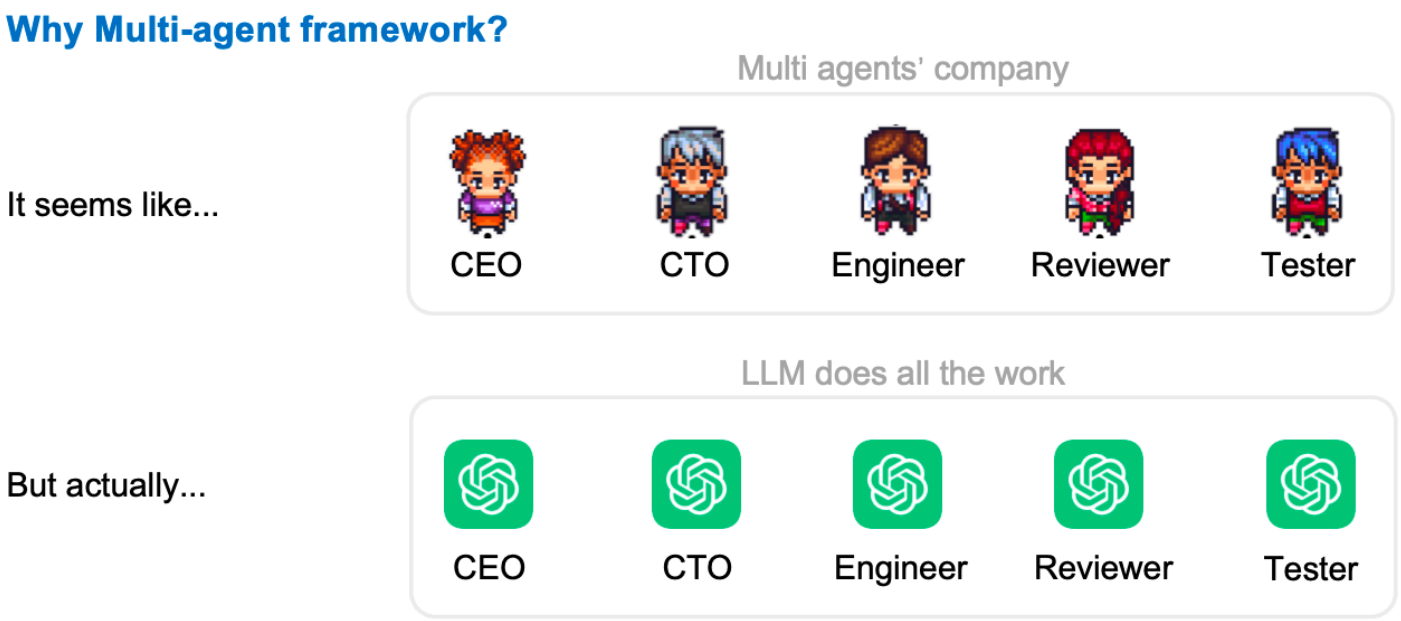
```
User: 追踪红色方块。
Agent -> detect_object("red block")
Tool  -> {"found": false}
Agent -> capture_image()
Tool  -> {"frame_id": 12}
Agent -> detect_object("red block")
Tool  -> {"found": true, "object_id": 3}
Agent -> start_tracking(3)
Tool  -> {"status": "tracking_started"}
```

相关的经典范式叫 **ReAct（Reasoning and Acting）**，强调推理、行动和外部观察交替进行。但 Agent Loop 不等于强化学习：循环中 Agent 根据新观察改变本次任务的行动，LLM 参数通常没有更新；强化学习则研究怎样利用经验更新策略，使以后表现更好。

3.4 Multi Agents

如果任务很大而且能够拆分，一个 Agent 可以把互相独立的子任务交给多个 Agent。例如“阅读三篇论文、选择适合机械臂追踪的方法、实现代码并完成测试”，可以分别安排研究、编程和测试 Agent，再由主 Agent 汇总结果。在人类世界的合作体系中，这种各司其职的分工风格似乎很容易理解。

—— **但既然这些 Agent 背后都是同一个 LLM，人为划分多个 Agent，扮演不同角色、承担不同任务，还有必要吗？**



Multi-Agent 最直观的两个价值是：

1. **并行（Parallelism）：**相互独立的子任务可以同时进行，例如三个 Agent 同时阅读三篇论文。
2. **上下文隔离（Context Isolation）：**每个 Agent 只接收完成自己任务所需的目标、资料和执行历史，减少无关信息干扰。

多个 Agent 可以共享同一个底层模型，也可以使用不同模型、提示词、工具和权限；它们也不一定并行，有依赖关系的子任务仍然需要按顺序执行。

多 Agent 会增加通信、协调和结果整合成本。如果多个任务都依赖同一份不断变化的程序状态，单 Agent 往往更简单；当任务能够清楚拆分、子任务相对独立且需要不同上下文时，多 Agent 更有价值。

四、课堂练习 & 课后作业

4.1 让机械臂模仿你的动作



操作步骤和验收要求在这份文档中的**任务二、三、四**：任务书：视觉感知、理解与多关节控制

4.2 把之前我们实现的功能作为你的 Agent 的 functions

这一任务的目标不是让 LLM 直接猜每个电机的控制量，而是把上节课已经实现的感知与控制能力封装为 Agent 可以选择的 Tools。建议至少定义三个接口，并先在仿真或 dry-run 模式中验证。

机器人工具接口示意

```python
def get_robot_state() -> dict:
    """返回关节角、追踪状态和错误状态。"""

def detect_object(description: str) -> dict:
    """根据文本描述寻找物体，返回 found 和 object_id。"""

def start_tracking(object_id: int) -> dict:
    """启动已经实现的视觉伺服追踪技能。"""

def stop_tracking() -> dict:
    """停止当前追踪任务。"""

# ...以上仅为示意，不是具体要求
```

除了上节课做的视觉伺服功能，你可以把更早我们完成的功能，包括运动到特定姿态、通过 IK 控制末端移动等等，都包装成可以被调用的工具。要求使用外部 LLM api call 来完成工具调用（而不是让你的 codex 调用一下就算做完了）。

验收内容：展示核心代码和 prompt，以及通过 LLM api 调用工具的流程。

选做：为这些工具编写一个最简 Skill，说明何时使用、调用前检查什么、何时必须停止等等。

4.3 实现视觉-语音端到端 Agent

—— 这就是我们本学期第一次课上，我给大伙儿演示的内容。当时我是说用 codex 可以造一个能陪你聊天的陪伴型机械臂，不知道大家还记不记得。

任务步骤：

1. 到 LLM 服务平台，例如智谱、轨迹流动、阿里云百炼等，选择一个多模态或全模态模型，例如 Qwen-omni；
2. 编写程序，获取机械臂腕部相机和电脑麦克风，作为模型的视觉和语音输入；
3. 编写 prompt 和 function calling 程序，让大模型可以通过你定义的 function 操控你的机械臂；

验收内容：展示你可以和你的机械臂通过语音对话、你的机械臂可以理解你所处的环境、你的 Agent 能够自主控制你的机械臂。